

UniConnecT — Team Presentation Guide

Team Mavericks · UIU · Dhaka · 2026–27

Members: Joydip Datta (Leader) · Md. Saem Ferdous · Md. Mahfujur Rahman Himel Akon · Md. Monabbur Hosen Bhuiyan · Md. Nazmul Hasan Nasim

This document maps every feature to a team contributor, explains how each system works end-to-end, and prepares the team for every question a teacher might ask during the presentation.

Table of Contents

1. [Project Overview](#)
 2. [System Architecture](#)
 3. [Tech Stack](#)
 4. [Five Contribution Areas](#)
 5. [Feature Deep-Dives \(How Everything Works\)](#)
 6. [Database Design](#)
 7. [Teacher Q&A Bank](#)
-

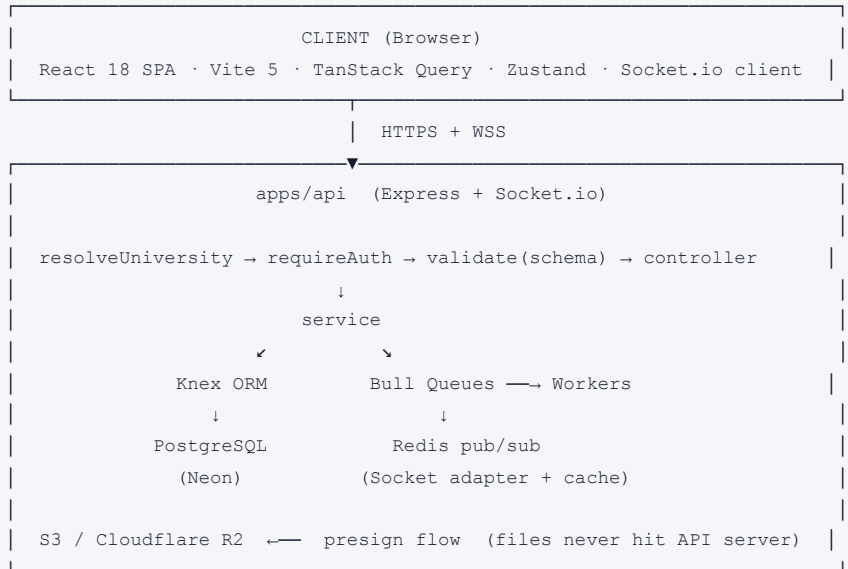
1. Project Overview

UniConnect is a private, multi-tenant university social network — think LinkedIn but locked to a single university's students, alumni, faculty, and admins. Each university gets its own isolated instance (tenant) sharing the same codebase and database cluster.

Metric	Value
Backend modules	21
Database migrations	81
Frontend pages	30+
React feature bundles	22
Background job queues	10
User roles	student, alumni, faculty, admin, driver
Lines of backend TypeScript	~15,000
Lines of frontend TypeScript/TSX	~18,000

Target users: Students (social feed, events, jobs), Alumni (mentorship, job posting), Faculty/Admin (content moderation, group management), Transport Staff/Driver (GPS shuttle broadcast).

2. System Architecture



Multi-tenancy: every DB table has `university_id` UUID FK.
No Postgres RLS – isolation enforced in service layer.
University resolved from `x-university-domain` HTTP header.

Monorepo:

- `apps/web` React SPA
- `apps/api` Express backend
- `packages/shared` shared Zod schemas, TypeScript types, socket event constants

Data flow for a typical POST /feed/posts:

Browser

- `axios` (injects `x-university-domain` + `Authorization: Bearer <token>`)
- Express router: `resolveUniversity → requireAuth → validate(createPostSchema)`
- `feedController.createPost`
- `feedService.createPost` (DB write via Knex)
- `notificationQueue.add(...)` (Bull enqueue – async, non-blocking)
- `getIo().to('uni:'+universityId).emit('NEW_POST', post)` (real-time)
- `sendSuccess(res, post, 201)`

3. Tech Stack

Layer	Technology	Why chosen
Frontend framework	React 18 + Vite 5	Fast HMR, lazy-loading via React.lazy
Styling	Tailwind CSS + CSS custom properties	Utility classes + design tokens without runtime overhead
State (server)	TanStack Query v5	Cache, background refetch, optimistic updates
State (client)	Zustand	Minimal boilerplate, no context drilling
Real-time	Socket.io	Rooms, auto-reconnect, fallback transports
Backend	Express + Node.js (TypeScript strict)	Mature ecosystem, team familiarity
Database	PostgreSQL (Neon cloud)	ACID, full-text search (tsvector/GIN), JSON columns
Query builder	Knex	Migrations + query building without ORM overhead
Queue/Cache	Redis + Bull	Reliable job processing, pub/sub for Socket.io scale
File storage	AWS S3 / Cloudflare R2	Presigned URL pattern; files never pass through API
Email	Resend API	Transactional email for OTPs and notifications
Deployment	Azure App Service (API) + Vercel (Web)	GitHub Actions CI/CD

Why Zustand for client state?

Think of Zustand as a lightweight global variables store. The alternative most people know is Redux — but Redux requires you to write "actions", "reducers", and "dispatchers" just to change one value. Zustand lets you do the same thing in a few lines with a normal function call. We use it only for state that truly needs to be global and does not come from the server:

Store	What it holds	Why global
authStore	Access token, current user object	Every page needs to know who is logged in
notificationsStore	Unread notification count	TopNav badge needs this without a fetch
themeStore	'dark' or 'light'	Every component reads the current theme
socketStore	The Socket.io client instance	Shared across all hooks/components
presenceStore	Map of <code>userId</code> → <code>online/offline</code>	Updated by socket events, read by avatars
pendingMsgsStore	Messages buffered while offline	Flushed to server on reconnect

```
// Example: authStore (apps/web/src/stores/authStore.ts)
const useAuthStore = create<AuthState>((set) => ({
  user: null,
  accessToken: null,
  setAuth: (user, token) => {
    set({ user, accessToken: token });
    localStorage.setItem('uc:has_session', '1'); // sentinel only — not the token
  },
  clearAuth: () => {
    set({ user: null, accessToken: null });
    localStorage.removeItem('uc:has_session');
  },
}));

// Access token lives in JS heap only — NOT in localStorage.
// localStorage holds only a boolean sentinel so the axios interceptor
// knows whether to attempt a silent refresh on 401.
// If the tab is closed, the token is gone — the refresh cookie handles recovery.
```

What Zustand does NOT hold: any data fetched from the API (posts, jobs, profiles, etc.). That is TanStack Query's job.

Why TanStack Query for server state?

TanStack Query (formerly React Query) solves the hardest part of frontend development: keeping data from the server in sync with what the user sees, without writing the same loading/error/cache/refetch boilerplate in every component.

Without it, every component that needs server data has to manually handle: loading spinner, error state, caching so two components don't double-fetch, re-fetching when data goes stale, and updating the UI after a mutation. With TanStack Query, all of this is automatic.

Problem	Without TanStack Query	With TanStack Query
Two components need the same data	Two separate API calls	Single cached fetch, both components share it
User edits their profile	Manually re-fetch + update state everywhere	<code>invalidateQueries(['profile'])</code> — done
Slow API → user sees stale data	Blank screen until response	Shows cached data immediately, refetches in background
Network drops mid-page	Manually handle retry logic	Automatic exponential backoff retry
Optimistic update (like before save)	Complex manual state juggling	Built-in <code>onMutate</code> / <code>onError</code> rollback

```
// Example: fetching posts (apps/web/src/features/feed/hooks/usePosts.ts)
const { data, isLoading, error } = useQuery({
  queryKey: ['feed', 'posts', { type, page }],
  queryFn: () => api.get('/feed/posts', { params: { type, page } }),
  staleTime: 30_000,          // treat as fresh for 30 seconds – no re-fetch
  gcTime: 5 * 60 * 1000,     // keep in cache for 5 minutes after component unmounts
});

// Example: creating a post mutation
const { mutate } = useMutation({
  mutationFn: (data) => api.post('/feed/posts', data),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['feed', 'posts'] });
    // ^ automatically re-fetches every feed query → UI updates everywhere at once
  },
});

// Rule enforced across the whole project:
//   Server data → useQuery / useMutation (TanStack Query)
//   UI/auth/socket state → Zustand store
//   NO useState for anything that comes from the API
```

4. Five Contribution Areas

Rule: Leader owns the most cross-cutting, architecturally critical work. Others own well-scoped feature verticals. Every member owns both backend (module) and frontend (feature bundle) for their area.

Team Roster

#	Name	Area	Headline responsibilities
1	Joydip Datta <i>(Leader)</i>	Core Architecture, Auth, Real-time & Admin	Monorepo, multi-tenancy, JWT/OTP auth, Socket.io+Redis, all Bull queues, admin panel, file upload, rate limiting, moderation, drafts, account deletion, design system, CI/CD
2	Md. Saem Ferdous	Social Feed, Groups & Notifications	Feed + hot_score ranking, post lifecycle, groups, notifications, news, badges, share
3	Md. Mahfujur Rahman Himel Akon	Jobs, Events, Connections, Profile & Onboarding	Jobs, events, connection graph, full profile suite, onboarding checklist, mentorship + points economy
4	Md. Monabbur Hosen Bhuiyan	Messaging, Presence & Push	Real-time chat, conversation types, Redis presence, privacy tiers, Web Push
5	Md. Nazmul Hasan Nasim	Campus Tools, Search & Content	Full-text search, explore, campus (lost-found + shuttle GPS), content-sync (WordPress + Skyvern), settings

Contributor 1 — Joydip Datta (LEADER): Core Architecture, Auth, Real-time & Admin

This is the hardest area. It underpins everything else. Nothing in the project works without this.

Owned systems:

System	Files / Location
Monorepo setup	pnpm-workspace.yaml, package.json root
Multi-tenancy middleware	apps/api/src/middleware/university.ts
JWT Auth (access + refresh)	apps/api/src/modules/auth/ · apps/api/src/services/token.service.ts
OTP verification	apps/api/src/services/otp.service.ts
Invite-based registration	apps/api/src/modules/auth/service.ts
Socket.io + Redis pub/sub	apps/api/src/socket/index.ts
All 10 Bull queues + workers	apps/api/src/queues/ · apps/api/src/workers/
Admin module (full)	apps/api/src/modules/admin/
Database schema (001–029)	apps/api/src/database/migrations/001-029_*.ts
Error handling middleware	apps/api/src/middleware/error-handler.ts
Response helpers	apps/api/src/utils/response.ts · errors.ts
Winston logger	apps/api/src/utils/logger.ts
Design system (tokens)	apps/web/src/styles/tokens.css
Theme switcher	apps/web/src/stores/themeStore.ts
CI/CD	.github/workflows/
File upload (presigned S3)	apps/api/src/modules/upload/ · apps/api/src/services/upload.service.ts
Rate limiting (Redis, per-user)	apps/api/src/middleware/rateLimiter.ts
Moderation (block / mute / report)	apps/api/src/modules/moderation/ · migration 078
Account deletion requests	apps/api/src/modules/users/ (deletion sub-API) · migration 079
Drafts (unified cross-domain)	apps/api/src/modules/drafts/ · migration 064 · apps/web/src/features/drafts/
Auth frontend	apps/web/src/features/landing/ · LoginPage.tsx · RegisterPage.tsx
Admin frontend	apps/web/src/pages/admin/ · apps/web/src/features/

How file upload works (leader must know this cold):

When a user wants to upload a profile photo or attach a file to a post, the browser first asks our server for a "permission slip" — a pre-signed URL. The server generates this short-lived URL (valid for 5 minutes) and sends it back. The browser then uploads the file directly to Amazon S3 (our cloud storage) using that URL — it never goes through our Express server at all. Once the upload is done, the browser just sends us the file's public link, and we save that link in the database. This way our server stays fast, never gets overloaded by large files, and we don't pay for bandwidth twice.

Presigned S3 PUT URL flow — file bytes never hit the API server:

1. Client: POST /api/upload/presign { filename, contentType }
2. API: upload.service.ts → S3.createPresignedPost() → returns { url, fields } (signed, 5 min TTL)
3. Client: PUT directly to S3/R2 URL (multipart form, bypasses Express entirely)
4. Client: sends resulting public URL to domain endpoint
e.g. PATCH /users/me { avatar_url: 'https://cdn.../abc.jpg' }
5. API: stores URL string in DB — no file bytes ever enter Node.js process

Why this matters: a 10 MB video upload doesn't block the Express event loop or consume Lambda/App Service memory. The API just signs a token and walks away.

How auth works (leader must know this cold):

UniConnecT is a closed network — you can't just sign up from the internet. An admin creates an invitation with a specific email and role (e.g. "this email is a student"). The new user uses that invite link, fills in their name and password, and we send them a one-time 6-digit OTP code by email to prove they own the address. Once they verify the OTP, their account is activated.

After that, every login gives back two tokens: a short-lived **access token** (15 minutes) stored in browser memory, and a long-lived **refresh token** (7 days) stored in an httpOnly cookie that JavaScript can never read. When the access token expires, the browser silently calls /auth/refresh, the server checks the cookie, and issues a new access token — all invisible to the user. If someone steals the access token, it's useless within 15 minutes. If they steal the cookie, the server can invalidate it in the database.

Account state scenarios (recently added — Joydip, commits 2a48a05 & 50056c2):

Before these fixes, certain edge cases caused confusing or broken behaviour. Here is every situation the auth system now handles, in plain words and then technically:

Scenario 1 — Login with correct password but email never verified. You registered, never typed the OTP, and now try to log in. Old behaviour: blocked with a generic error, OTP already expired — stuck forever. New behaviour: the server sees the password is correct, revokes the old expired OTP, generates a brand new one, sends it to your email right away, and sends back ACCOUNT_NOT_VERIFIED (403). The frontend catches that code and redirects you straight to the OTP page. You verify the fresh code and you're in.

Scenario 2 — Re-registering with an email that has a soft-deleted AND unverified account. An admin invited someone, they started registration, never verified, and the admin later deleted that ghost account. The email is still in the database (soft-deleted). If that person tries to register again, the old code would crash with a silent 500 (duplicate key error on the global unique index). New behaviour: the server detects the email row is both is_deleted=true AND is_verified=false (a ghost/abandoned account), hard-deletes that row entirely, and lets registration proceed cleanly.

Scenario 3 — Re-registering with an email that has a soft-deleted but VERIFIED account. The admin deleted a real, active account (e.g. a student who graduated). That person tries to register fresh. We do NOT allow it — their old data exists and an admin action is needed to restore it properly. The server returns ACCOUNT_DEACTIVATED (409) with the message "Please contact your university admin to restore access." The frontend shows this message clearly instead of a generic error.

Scenario 4 — Logging in to a self-deactivated account. A user deliberately deactivated their own account via Settings → Account → Deactivate. Their row has is_active=false AND deactivated_at is set (this is the signal it was self-initiated, not admin-forced). When they log in again with the correct password, the server automatically reactivates the account (is_active=true, deactivated_at=null) and logs them in normally. No admin needed — the user changed their mind.

Scenario 5 — Logging in to an admin-disabled account. An admin forcibly disabled an account (is_active=false, but deactivated_at is NULL — the signal it was admin-initiated). The server returns the same generic "Invalid email or password" (401) — it deliberately does not say "your account is disabled" to avoid leaking information. Only the admin can re-enable it.

Registration:

```
POST /auth/register (body: token, email, password, name)
  → validate invite token exists + email matches
  → bcrypt.hash(password)
  → insert user + profile in transaction
  → otpService.generate(userId, 'email_verify') → store in Redis (TTL 10 min)
  → emailQueue.add({ to, otp }) → email.worker sends via Resend API
  → return { message: 'OTP sent' }
```

POST /auth/verify-otp

```
  → otpService.verify(userId, otp, purpose)
  → set users.is_verified = true
  → return { accessToken, user } + Set-Cookie: refreshToken (httpOnly, 7d)
```

Login:

```
POST /auth/login
  → bcrypt.compare(password, hash)
  → tokenService.signAccess({ userId, universityId, role }) (JWT 15 min)
  → tokenService.signRefresh() → 256-bit random → stored in user_sessions
  → Set-Cookie: refreshToken (httpOnly)
```

Silent refresh (frontend interceptor):

```
  axios 401 interceptor → check localStorage 'uc:has_session' sentinel
  → POST /auth/refresh (cookie sent automatically)
  → new accessToken → retry original request
```

Account state decision tree on Login (auth/service.ts):

```
user not found                → 401 (same message as wrong password – no leaking)
is_active=false AND deactivated_at=NULL → 401 (admin-disabled, generic message)
password wrong                → 401
is_verified=false (correct password) → revoke old OTP → generate fresh OTP → email it
                                   → 403 ACCOUNT_NOT_VERIFIED
                                   → frontend navigates to /verify-otp?purpose=verify
is_active=false AND deactivated_at SET → reactivate (is_active=true, deactivated_at=null)
                                   → login succeeds (self-deactivation = user changed mind)
all checks pass                → issue access token + refresh token → success
```

Account state decision tree on Register (auth/service.ts):

```
email exists, is_deleted=true, is_verified=false → hard-delete ghost row → proceed with register
email exists, is_deleted=true, is_verified=true   → 409 ACCOUNT_DEACTIVATED ("contact admin")
email exists, is_deleted=false                   → 409 CONFLICT ("account already exists")
race condition duplicate key                     → caught → 409 CONFLICT (no 500 leak)
```

How the database is handled for Auth (Joydip, leader must know):

The auth system touches several tables and keeps sensitive data out of the JWT. Here is exactly what lands in the database and why:

Tables involved in auth:

```
users          - stores hashed password (bcrypt, 12 rounds), is_verified, is_active,
                 deactivated_at, role, username (unique per university)
user_sessions  - one row per active refresh token
                 (token_hash, user_id, expires_at, user_agent, ip_address)
invitations    - invite tokens (token, email, role, used_at) - consumed on register
university_audit_log - admin actions recorded here (role changes, disables, etc.)
```

Key design decisions:

- Refresh tokens are 256-bit random strings, hashed (SHA-256) before storage – the database never holds the raw token, only the hash.
If the DB is breached, stolen hashes cannot be replayed.
- Passwords: bcrypt with cost factor 12 → ~250ms per compare – slow enough to make brute-force impractical, fast enough for UX.
- JWT payload: only { userId, universityId, role } – no email, no name.
Minimises data exposure if a token is decoded client-side.
- OTPs are NOT stored in Postgres – they live in Redis with a TTL key:
otp:{purpose}:{userId} → { code: '384920', attempts: 0 }
Expiry is automatic via Redis TTL (10 min). No cleanup job needed.
- is_deleted = soft delete: the row stays (referenced by posts, comments, etc.) but auth rejects login. The ghost-user logic in Scenario 2 hard-deletes ONLY unverified soft-deleted rows (no content to preserve).

Migration trail for auth:

```
001 - create universities
002 - create users (core columns + bcrypt hash)
003 - create user_sessions (refresh token store)
004 - create invitations
067 - add users.deactivated_at (self-deactivation signal)
072 - add users.username (per-university unique vanity slug)
```

ACTUAL QUERIES (file: apps/api/src/modules/auth/service.ts):

```
-- Ghost-row detection + hard-delete (line 121-129):
const existingUser = await db<UserRow>('users')
  .whereRaw('lower(email) = ?', [email.toLowerCase()])
  .first()
// if is_deleted AND !is_verified → hard delete so re-registration works:
await db('users').where({ id: existingUser.id }).delete()

-- New user insert inside a transaction (line 146-157):
user = await db.transaction(async (trx) => {
  const [createdUser] = await trx<UserRow>('users')
    .insert({
      university_id: universityId,
      username,
      email,
      password_hash: passwordHash, // bcrypt hash, NOT raw password
      role,
      is_verified: false,
    })
    .returning('*')
  await trx('profiles').insert({ user_id: createdUser.id, full_name: name, ... })
})

-- Mark account verified after OTP (line 217):
await db('users').where({ id: user.id }).update({ is_verified: true })
```

```
-- Self-deactivated reactivation on next login (line 276):
await db('users').where({ id: user.id })
  .update({ is_active: true, deactivated_at: null })
```

How moderation & rate limiting work (leader-owned safety layer):

Moderation gives every user three self-protection tools. If someone is harassing you, you can **block** them — they vanish from your feed and can no longer message you (and you vanish from theirs too). If you just want to stop seeing someone's posts without them knowing, you **mute** them — it's one-sided and silent. If someone posts harmful content, you **report** it — that report goes into the admin panel for a faculty/admin to review and act on.

Rate limiting is a spam prevention layer. Every time someone clicks a button that writes data (posting, blocking, reporting, etc.), our server counts how many times that user has done it in the last time window. If they exceed the limit, we return a "429 Too Many Requests" error. We count by **user ID** rather than IP address — that way a whole university behind a shared WiFi router doesn't accidentally get blocked together.

```
Moderation module (user-level trust & safety, not just admin):
  POST /moderation/block/:userId - hide a user both ways (no posts, no DMs)
  DELETE /moderation/block/:userId - unblock
  POST /moderation/mute/:userId - silence a user (you stop seeing them; they don't know)
  DELETE /moderation/mute/:userId - unmute
  POST /moderation/report - file a content/user report → lands in admin queue
  GET /moderation/blocks · /mutes - manage your lists
→ backed by user_moderation table (migration 078)
→ admin resolves reports in the admin panel (university_audit_log records the action)

Rate limiting (rateLimiter.ts - Redis sliding counter):
  key = rl:{prefix}:{subject} where subject = u:{userId} (authed) or ip:{ip} (anon)
→ INCR key with a TTL window; reject with 429 (tooManyRequests) over the limit
→ keyed by USER id, not IP, so a whole university behind one NAT isn't throttled as one client
→ writeLimiter guards all mutating routes (block, mute, report, post, etc.)
→ disabled in NODE_ENV=test (noop) so tests aren't throttled
```

Contributor 2 — Md. Saem Ferdous: Social Feed, Groups & Notifications

Owned systems:

System	Files / Location
Feed (posts, reactions, comments, polls)	apps/api/src/modules/feed/
Feed ranking (hot_score)	apps/api/src/workers/feed-ranking.worker.ts · migration 071
Post lifecycle (draft→scheduled→published→archived)	apps/api/src/workers/post-lifecycle.worker.ts · migration 072
Groups (join-request, roles, resources, study sessions)	apps/api/src/modules/groups/
Group digest (weekly Bull cron)	apps/api/src/workers/group-digest.worker.ts
Notifications	apps/api/src/modules/notifications/
News module	apps/api/src/modules/news/
Tags / hashtags	migration 013 · preprocessHashtags.ts
Share feature	apps/web/src/features/share/
Badge system	apps/api/src/workers/badge.worker.ts
Feed frontend	apps/web/src/features/feed/
Groups frontend	apps/web/src/features/groups/
Notifications frontend	apps/web/src/features/notifications/
News frontend	apps/web/src/features/news/

How feed ranking works:

The feed has two modes: **Recent** (newest posts first — simple) and **Top** (most interesting posts first — the hard part). For "Top", we use a scoring formula inspired by Hacker News. A post that gets 50 reactions in the first hour beats a post that got 50 reactions over a week. The score decays over time — old posts sink down naturally. We store this score in a column called `hot_score` that a background job recalculates every hour. This way, when you open the feed, the database just sorts by that pre-calculated number — no slow calculation happens while you wait.

Columns on posts table (migration 071):

```

reaction_count  - incremented transactionally on every reaction write
comment_count   - incremented transactionally on every comment write
hot_score       - precomputed Hacker News-style score

```

Hot score formula (feed-ranking.worker.ts):

```

score = (reactions + comments * 2) / ((age_hours + 2) ^ 1.8)

```

```

→ Bull cron runs every hour → recalculates hot_score for all posts
→ "Top" feed sorts on idx_posts_hot_score index (no subquery cost)
→ "Recent" feed sorts on created_at DESC

```

Post lifecycle (post-lifecycle.worker.ts):

```

→ Bull cron every minute scans posts
→ publish_at in past + is_published=false → flip to published + emit SOCKET event
→ archived_at in past + is_published=true → flip to archived
→ States: draft (no publish_at, is_published=false)
           scheduled (publish_at in future)
           published (is_published=true)
           archived (archived_at in past)

```

How groups work:

Groups work like Facebook groups but for a university. A group can be **public** (anyone can join instantly), **private** (you have to request to join and a group admin approves you), or **restricted** (invitation only). Inside a group, members have different roles — admin, moderator, or regular member — each with different permissions. Groups also support extra features: file links (resources) that members can share, and study sessions where members can RSVP to attend. Posts can be pinned by group admins so important announcements stay at the top.

```
Group types: public, private, restricted + system_group flag
Private join flow:
  POST /groups/:id/join →
    if public: insert member immediately
    if private: insert group_join_requests (status='pending')
                  → notify group admins
  PATCH /groups/:id/join-requests/:reqId (admin only):
    status='approved' → insert into group_members
    status='rejected' → update join request

Member roles: admin > moderator > member
Resources: file links stored in group_resources (view tracking via group_resource_views)
Study sessions: group_study_sessions table + RSVP via group_session_rsups
```

How the database is handled for Feed Ranking (Saem must know):

The feed ranking system avoids expensive real-time computation by pre-writing scores into the database. Here is the full database picture:

posts table columns relevant to ranking (migration 071):

```
reaction_count  INTEGER NOT NULL DEFAULT 0
comment_count   INTEGER NOT NULL DEFAULT 0
hot_score       DOUBLE PRECISION NOT NULL DEFAULT 0
publish_at      TIMESTAMPTZ           -- scheduled publish
archived_at     TIMESTAMPTZ           -- auto-archive deadline
is_published    BOOLEAN DEFAULT false
```

Index that makes "Top" feed fast:

```
CREATE INDEX idx_posts_hot_score ON posts(university_id, hot_score DESC)
→ ORDER BY hot_score DESC becomes an index scan, not a table scan
```

How counters stay accurate:

```
→ React write (reaction INSERT) runs in a Knex transaction:
  BEGIN;
  INSERT INTO reactions (post_id, user_id, type) VALUES (?, ?, ?);
  UPDATE posts SET reaction_count = reaction_count + 1 WHERE id = ?;
  COMMIT;
→ Same for comments (comment_count).
→ Because it's in the same transaction, count is always consistent –
  no separate sync job needed, no race condition.
```

Feed-ranking Bull cron (every hour):

```
→ SELECT id, reaction_count, comment_count, created_at FROM posts
  WHERE is_published = true AND university_id = ?
→ Computes hot_score in JS: (reactions + comments*2) / ((ageHours+2)^1.8)
→ Batch UPDATE posts SET hot_score = ? WHERE id = ?
→ Only updates rows whose score changed by >0.01 (avoids unnecessary I/O)
```

Post lifecycle Bull cron (every minute):

```
→ SELECT id FROM posts WHERE publish_at <= now() AND is_published = false
→ UPDATE posts SET is_published = true WHERE id IN (...)
→ Separate query for archiving: archived_at <= now() AND is_published = true
```

Why store hot_score in DB instead of computing at query time?

Computing power(age, 1.8) for every row in every feed request would be
O(n) CPU work per request. Storing it means feed load = pure index seek.

ACTUAL QUERIES (file: apps/api/src/modules/feed/service.ts):

```
-- Reaction upsert (idempotent – same user reacting again just changes type)
-- (line 531-544):
await db('reactions')
  .insert({
    user_id: context.userId,
    target_id: postId,
    target_type: 'post',
    reaction_type: reactionType,
  })
  .onConflict(['user_id', 'target_id', 'target_type'])
  .merge({ reaction_type: reactionType, created_at: db.fn.now() })

await syncPostCounters(postId)
// syncPostCounters does:
//   UPDATE posts
//   SET reaction_count = (SELECT COUNT(*) FROM reactions WHERE target_id = postId AND target_type='post'),
//       comment_count  = (SELECT COUNT(*) FROM comments WHERE post_id = postId)
```

```
// WHERE id = postId

-- Notify post author (deduplicated - only fires ONCE per actor per post)
-- (line 558-561):
const [{ count }] = await db('notifications')
  .where({ user_id: post.author_id, type: 'post_reaction',
    actor_id: context.userId, reference_id: postId })
  .count<{ count: string }[]>({ count: '*' })
if (Number(count) > 0) return // skip - already notified
```

Contributor 3 — Md. Mahfujur Rahman Himel Akon: Jobs, Events, Connections, Profile & Onboarding

Owned systems:

System	Files / Location
Jobs (listings, applications, saved)	apps/api/src/modules/jobs/
Events (create, RSVP, group-linked)	apps/api/src/modules/events/
Connections (bidirectional graph)	apps/api/src/modules/connections/ · migration 051
Profile (extended sections)	apps/api/src/modules/users/ (profile sub-API)
Experience entries	table profile_experiences · migration 053
Education entries	table profile_education · migration 054
Featured items	table profile_featured · migration 055
Profile analytics & viewers	table profile_views · migration 057
Resume export	apps/web/src/features/profile/ResumeExportButton.tsx
Profile vanity URLs	users.username unique per university · migration 072
Onboarding checklist & profile progress	GET /users/me/progress · profileProgressSchema · apps/web/src/features/onboarding/
Mentorship module	apps/api/src/modules/mentorship/
Mentorship points economy	POINTS_PER_SESSION=10 · POINTS_PER_USD=100
Gift card redemption	mentorship_redemptions table
Jobs frontend	apps/web/src/features/jobs/
Events frontend	apps/web/src/features/events/
Connections frontend	apps/web/src/features/connections/
Profile frontend	apps/web/src/features/profile/
Mentorship frontend	apps/web/src/features/mentorship/

How connections work:

Connections are like LinkedIn's "Connect" feature — both people have to agree. You send a request, the other person accepts, and then you're connected. We originally had a one-way "follow" system (like Twitter), but replaced it entirely with this two-way connection graph. Why? Because for a professional university network, mutual connections matter — they show trust. Once connected, you can see each other's profiles, send direct messages, and appear in each other's "People You May Know" suggestions. The app also detects **mutual connections** — people who are connected to both of you — to help you expand your network.

Old model: follows table (one-directional, dropped in migration 052)

New model: connections table (bidirectional LinkedIn-style)

```
connections(id, university_id, requester_id, recipient_id, status, note)
status: 'pending' | 'accepted'
```

Flow:

```
POST /connections/request/:userId
  → insert connection(requester=me, recipient=target, status='pending')
  → emit CONNECTION_EVENTS.REQUEST_RECEIVED to user:{targetId} room
```

```
POST /connections/:connectionId/accept
  → update status='accepted'
  → emit CONNECTION_EVENTS.ACCEPTED to user:{requesterId} room
```

```
GET /connections/mutual/:userId
  → find users connected to BOTH me AND target userId
  → two-hop intersection query
```

Connection status on profiles:

```
getUser() checks connections table → returns 'none' | 'pending_sent' |
'pending_received' | 'connected' so the ConnectButton renders correctly
```

ACTUAL QUERIES (file: apps/api/src/modules/connections/service.ts):

```
-- Check for any existing relationship in EITHER direction before inserting
-- (line 68-76):
const existing = await db('connections')
  .where(function () {
    this.where({ requester_id: currentUserId, addressee_id: targetUserId })
      .orWhere({ requester_id: targetUserId, addressee_id: currentUserId })
  })
  .andWhere('university_id', universityId)
  .first<ConnectionRow>()
// This prevents duplicate rows — one query covers both directions.

-- Insert a new pending connection (line 82-90):
const [connection] = await db('connections')
  .insert({
    requester_id: currentUserId,
    addressee_id: targetUserId,
    university_id: universityId,
    note: note ?? null,
    status: 'pending',
  })
  .returning('*')

-- Accept: flip status to 'accepted' (line 150-153):
const [updated] = await db('connections')
  .where({ id: connectionId })
  .update({ status: 'accepted', updated_at: db.fn.now() })
  .returning('*')

-- Connection status CASE expression injected into every getUser() query
-- (file: apps/api/src/modules/search/service.ts, line 176-185):
db.raw(`
CASE
```

```

    WHEN c.id IS NULL THEN 'none'
    WHEN c.status = 'accepted' THEN 'connected'
    WHEN c.requester_id = ? AND c.status = 'pending' THEN 'pending_sent'
    WHEN c.addressee_id = ? AND c.status = 'pending' THEN 'pending_received'
    ELSE 'none'
END AS "connectionStatus",
[requesterId, requesterId]
)
-- This raw CASE runs inside a LEFT JOIN on the connections table so
-- the ConnectButton gets the right label in a single query, no extra round-trip.

```

How mentorship points economy works:

Students can request mentorship from alumni. Alumni set how many students they can take on at a time (default 3). When a student sends a request with a note explaining what they need, the alumni gets notified and can accept or decline. Once accepted, a private chat thread is automatically created for them to communicate.

To reward alumni for their time, we have a points system. After each completed mentorship session, the alumnus earns 10 points. When they accumulate enough points, they can redeem them for gift cards (e.g. \$10 Amazon card = 1000 points). Admins upload the actual gift card codes and mark redemptions as fulfilled. It's a lightweight incentive system to keep alumni engaged in helping students.

```

Every completed mentorship session → +10 points to mentor (alumni)
Points can be redeemed for gift cards:
  gift_cards table: vendor, title, threshold_points, value_usd_cents

POST /mentorship/redeem
  → check user.mentorship_points >= gift_card.threshold_points
  → deduct points → insert mentorship_redemptions
  → admin fulfills by adding code_text + marking fulfilled

Capacity: alumni sets max_mentees on profile (default 3)
Accept request → count active mentees → reject if at capacity

```

How onboarding / profile progress works:

When a new student first logs in, they're shown an onboarding checklist — a simple to-do list like "Add a profile photo", "Write a bio", "Add your experience", "Make your first post". Each item links directly to where you need to go to complete it. The backend tracks which steps are done by checking the actual database — if your `avatar_url` column is empty, `hasAvatar` is false, and the checklist shows that step as incomplete. As you complete each step, a progress score (0–100) increases. Once you hit 100, the checklist disappears and you're considered fully onboarded.

```

GET /users/me/progress → profileProgressSchema (packages/shared)
Returns first-run signals that drive the onboarding checklist:
  profileScore      0-100 weighted completion of profile fields
  isVerified        email OTP completed
  hasAvatar / hasBio / hasHeadline  granular per-field flags
  hasAddedExperience / hasAddedEducation
  hasMadePost       has published at least one post
  connectionCount   number of accepted connections

Frontend: OnboardingChecklist.tsx reads these flags and deep-links the user
straight to the exact gap (e.g. "Add a bio" → opens EditProfileModal).
The checklist disappears once profileScore hits 100.

```

Contributor 4 — Md. Monabbur Hosen Bhuiyan: Messaging, Real-time Presence & Push Notifications

Owned systems:

System	Files / Location
Messaging (DMs, group chat, mentorship threads)	apps/api/src/modules/messages/
Conversation types (direct, group, mentorship)	conversations.type discriminator
Presence (Redis-based online status)	apps/api/src/modules/presence/
Privacy tiers for visibility	apps/web/src/features/settings/PrivacySection.tsx
Web Push (VAPID subscriptions)	apps/api/src/modules/push/ · migration 066
Push worker (fan-out)	apps/api/src/workers/push.worker.ts
Notification digest (Bull cron)	apps/api/src/workers/notification-digest.worker.ts
Pending messages store (offline queue)	apps/web/src/stores/pendingMsgsStore.ts
Socket store	apps/web/src/stores/socketStore.ts
Presence store	apps/web/src/stores/presenceStore.ts
Messages frontend	apps/web/src/features/messages/
Presence frontend	apps/web/src/features/presence/

How real-time messaging works:

When you open a conversation and type a message, it travels through a persistent WebSocket connection — not a regular HTTP request. This means the other person sees your message appear instantly without refreshing the page. We use Socket.io which organises these connections into "rooms" — a conversation room, a personal room (for your own notifications), and a university-wide room (for feed updates). When your message is sent, the server saves it to the database first, then immediately broadcasts it to everyone in that conversation's room. If the other person is offline, their next login will load all missed messages from the database. We also handle the case where your internet drops mid-message — messages are buffered locally and flushed when you reconnect.

There are three types of conversations: **direct** (one-on-one DM), **group** (group chat), and **mentorship** (automatically created when a mentorship request is accepted, linking the student and alumni permanently).

Socket.io rooms:

```
uni:{universityId}    - broadcast to whole university (feed posts, events)
user:{userId}         - personal (notifications, connection requests)
conv:{conversationId} - chat messages
```

Message flow:

Client sends: `socket.emit('SEND_MESSAGE', { convId, content })`

Server:

- verify user is participant in conversation
- INSERT into messages table
- `io.to('conv:'+convId).emit('NEW_MESSAGE', message)`
- `notificationQueue.add()` for offline participants

Offline messages:

`pendingMsgsStore` buffers outgoing messages when socket disconnected
→ on reconnect, flushes buffer → server deduplicates by `client_id`

Mentorship conversations:

When mentorship request accepted:

- INSERT into conversations(`type='mentorship', mentorship_request_id=X`)
- Both parties auto-joined to conversation
- `mentorship_requests.conversation_id` FK set

ACTUAL QUERIES (file: `apps/api/src/modules/messages/service.ts`):

```
-- Privacy gate: DMs only allowed between connected users (line 279-292):
const connection = await db('connections')
  .where(function () {
    this.where({ requester_id: context.userId, addressee_id: other.userId })
      .orWhere({ requester_id: other.userId, addressee_id: context.userId })
  })
  .andWhere('status', 'accepted')
  .andWhere('university_id', context.universityId)
  .first()
if (!connection) throw forbidden('You must be connected to message this person')

-- Save message to DB inside a transaction, then broadcast via Socket.io
-- (line 303-317):
const messageId = await db.transaction(async (trx) => {
  const [message] = await trx('messages')
    .insert({
      conversation_id: convId,
      sender_id: context.userId,
      content: input.content ?? null,
      media_urls: input.media_urls,
      reply_to_id: input.reply_to_id ?? null,
      type: input.type,
    })
    .returning<{ id: string }[]>('id')
  return message.id
})
// After the transaction commits:
io.to(`conv:${convId}`).emit('message:new', mapped) // real-time delivery
io.to(`conv:${convId}`).emit('conversation:updated', ...) // update last-message preview

-- Edit a message (line 342):
await db('messages')
```

```
.where({ id: msgId, conversation_id: convId })
.update({ content: input.content })
// Only the sender can edit - checked before this line (sender_id === context.userId)
```

How presence works:

The green dot next to a user's name showing they're "online" is powered entirely by Redis (an in-memory database) — we never write online status to our main PostgreSQL database because it would get millions of tiny writes per day. Instead, when you connect, we increment a counter in Redis. When you open a second tab, the counter goes to 2. Close one tab — counter drops to 1, still online. Close the last tab — counter hits 0, you're marked offline, and we update your `last_seen` time in the database once. Every 30 seconds your browser sends a "ping" to keep the Redis counter alive. If we stop receiving pings (e.g. laptop closed), the counter expires automatically and you appear offline. Users can also choose their privacy: show online status to everyone, only connections, or nobody.

```
Redis keys:
  presence:count:{userId}      - socket count (int, TTL refreshed by heartbeat)
  presence:online:{universityId} - SET of online user IDs

Connect (0→1 transition):
  socket.on('connection') → registerConnect(userId, universityId)
  → INCR presence:count:{userId}
  → if result === 1:  SADD presence:online:{universityId}, userId
                     → broadcastPresence(userId, 'online')
                     emit PRESENCE_EVENTS.ONLINE to uni room

Heartbeat:
  Client emits PRESENCE_EVENTS.PING every 30s
  → server: EXPIRE presence:count:{userId} 120  (refresh TTL)

Disconnect (1→0 transition):
  registerDisconnect → DECR count
  → if result === 0:  SREM online set
                     UPDATE users.last_seen = now()
                     emit PRESENCE_EVENTS.OFFLINE

Privacy tiers (OnlineVisibilityTier):
  everyone | connections_only | nobody
  → presence.service enforces tier when returning GET /presence
```

Contributor 5 — Md. Nazmul Hasan Nasim: Campus Tools, Search, Content-Sync & Settings

Owned systems:

System	Files / Location
Full-text search (GIN indexes)	apps/api/src/modules/search/ · migration 070
pg_trgm fuzzy search fallback	migration 024
Explore module (trending, discover)	apps/api/src/modules/explore/
Campus module (lost-and-found + shuttle)	apps/api/src/modules/campus/
Shuttle GPS broadcast (driver role)	POST /shuttle/locations · apps/web/src/pages/ShuttleDrivePage.tsx
Content-sync (WordPress + Skyvern)	apps/api/src/modules/content-sync/
Content-sync worker	apps/api/src/workers/content-sync.worker.ts
Skyvern browser-automation fallback	apps/api/src/services/skyvern.service.ts
Settings (notification + privacy JSONB)	apps/api/src/modules/users/ (settings sub-API) · migration 065
Search frontend	apps/web/src/features/search/
Campus / lost-found / shuttle frontend	apps/web/src/features/lost-found/ · shuttle/
Settings frontend	apps/web/src/features/settings/
Content-sync frontend (admin)	apps/web/src/features/content-sync/

How full-text search works:

When you type "machine learning" in the search bar and results appear in milliseconds, that's not a simple text comparison — it's PostgreSQL's built-in full-text search. Every time a post is written or a profile is updated, the database automatically computes a special compressed representation of all the text (called a `tsvector`) and stores it in a hidden column. We then build a GIN index on top of it — think of it like a book's index that maps every word to the pages it appears on. When you search, the query looks up the index directly instead of scanning every row. Results are ranked by relevance — a match in someone's name counts more than a match buried in their bio. If the main search finds nothing, we fall back to a "fuzzy" search that can handle typos.

Migration 070 adds STORED generated columns on each searchable table:

```
posts.search_vector    TSVECTOR GENERATED ALWAYS AS (  
    setweight(to_tsvector('english', content), 'C')  
) STORED  
  
profiles.search_vector TSVECTOR GENERATED ALWAYS AS (  
    setweight(to_tsvector('simple', full_name), 'A') ||  
    setweight(to_tsvector('english', COALESCE(headline, '')), 'B') ||  
    setweight(to_tsvector('english', COALESCE(bio, '')), 'C')  
) STORED
```

GIN index on each search_vector column → O(1) lookup instead of O(n) scan

Search query:

```
GET /api/v1/search?q=machine+learning&type=posts,users  
→ service builds: WHERE search_vector @@ plainto_tsquery('english', ?)  
→ ORDER BY ts_rank(search_vector, query) DESC  
→ fallback: WHERE full_name ILIKE '%?%' (pg_trgm)
```

Multi-entity search returns typed results:

```
{ users: [...], posts: [...], jobs: [...], events: [...], groups: [...] }
```

ACTUAL QUERIES (file: apps/api/src/modules/search/service.ts):

```
-- People search base query with privacy + moderation filters (line 130-149):  
const baseQuery = db('profiles as p')  
    .join('users as u', 'u.id', 'p.user_id')  
    .where('u.university_id', universityId)  
    .where('u.is_active', true)  
    // Exclude users who opted out of discovery (privacy JSONB check):  
    .whereNotExists(function () {  
        this.select(db.raw('1'))  
        .from('user_settings')  
        .whereRaw('user_settings.user_id = u.id')  
        .whereRaw("user_settings.privacy_preferences->>'discoverable' = 'false'")  
    })  
    // Exclude blocked users in either direction:  
    .whereNotExists(function () {  
        this.select(db.raw('1'))  
        .from('user_blocks as ub')  
        .whereRaw(  
            '(ub.blocker_id = ? AND ub.blocked_id = u.id) OR '  
            '(ub.blocked_id = ? AND ub.blocker_id = u.id)',  
            [requesterId, requesterId],  
        )  
    })  
  
-- Apply full-text search when query string present (line 151):  
if (filters.q) applyTextSearch(baseQuery, 'p.search_vector', 'p.full_name', filters.q)  
// applyTextSearch adds:  
//   WHERE p.search_vector @@ plainto_tsquery('english', ?)  
//   OR p.full_name ILIKE '%?%' (pg_trgm fuzzy fallback)  
  
-- Order by ts_rank relevance score (line 191):  
if (filters.q) {  
    applyTextOrder(rowsQuery, 'p.search_vector', 'p.full_name', filters.q, 'p.full_name ASC')  
}
```

```
// applyTextOrder adds:
//   ORDER BY ts_rank(p.search_vector, plainto_tsquery('english', ?)) DESC
// Without a query, falls back to alphabetical: ORDER BY p.full_name ASC
```

How content-sync works:

Universities often post news and notices on their own websites (usually WordPress). Instead of admin staff manually copying content into UniConnect, we built an automated importer. An admin pastes in the URL of the university's WordPress news/events/notices feed, and with one click we pull all recent articles directly from the WordPress API. The articles land in an "Inbox" for the admin to review and publish. We never auto-publish — someone always approves first.

If the WordPress API isn't available (some university websites are plain HTML with no API), we have a backup plan: we send a real browser (using Skyvern automation) to visit the website, read the HTML like a human would, extract the content, and bring it back. This is slower (can take a minute) but works on any website. Every import is idempotent — running it twice never creates duplicate articles.

```
Admin configures WordPress REST API URLs per category (news/notices/events)
  PATCH /api/v1/admin/content-sync/config → university_settings updated

POST /api/v1/admin/content-sync/run
→ contentSyncQueue.add({ universityId })
→ content-sync.worker picks up job:
  1. Fetch WordPress REST API: GET {url}?per_page={entriesPerSource}
  2. Parse JSON → extract title, content, date, featured_image
  3. If WP fetch fails → skyvern.service.ts fallback:
      Skyvern browser automation: navigate URL, extract DOM, return structured data
  4. Deduplicate by external_id (idempotent)
  5. INSERT into news/events table with is_published=false
  6. Emit CONTENT_SYNC_EVENTS.COMPLETED via socket
  7. Log run in content_sync_runs table

Admin reviews imported items at GET /api/v1/admin/content-sync/pending
→ publishes selected items → visible to students
```

5. Feature Deep-Dives

5.1 Multi-tenancy

Imagine one app that can serve UIU, BRAC University, and NSU at the same time — each thinking it has its own private social network. That's multi-tenancy. We use a single database and a single codebase. Every piece of data (every post, every user, every event) has a `university_id` tag attached to it. When a request comes in from the UIU app, the server reads the `x-university-domain: uiu.ac.bd` header, looks up UIU's ID, and from that point every database query automatically includes `WHERE university_id = 'UIU's UUID'`. It's physically impossible for a UIU student to accidentally see a BRAC University post because every query is filtered before any data is returned.

```
Every domain table: university_id UUID FK + indexed
Request lifecycle:
  1. Client sends x-university-domain: uiu.ac.bd header
  2. resolveUniversity middleware: SELECT * FROM universities WHERE domain = ?
  3. Sets req.university = { id, name, domain }
  4. All service calls receive universityId from req.university.id
    NEVER from req.body (security: prevents tenant hopping)

University data is fully isolated at query level:
db('posts').where({ university_id: universityId, ... })
```

5.2 File Upload (Presign Flow)

File bytes never travel through the API server — critical for performance.

```
1. Client: POST /api/upload/presign { filename, contentType }
2. API: S3.createPresignedPost() → returns { url, fields } (signed, expires 5 min)
3. Client: PUT directly to S3/R2 URL (multipart form, bypasses API)
4. Client: sends the resulting public URL to the domain endpoint
   e.g. PATCH /users/me { avatar_url: 'https://cdn.../abc.jpg' }
5. API: stores URL in DB — no file bytes ever hit Node.js process
```

5.3 Background Jobs (Bull + Redis)

Imagine you post something and the server needs to: send notification emails to your followers, award you a badge, update the leaderboard, and schedule your post to be published tomorrow. Doing all of that while the user waits for the "Post" button to respond would be very slow. Instead, we immediately save the post and respond with success. Then, separately, a queue of background jobs handles all the slow work. Think of it like a restaurant — the waiter takes your order instantly (fast response to you) and passes a ticket to the kitchen (background worker does the actual cooking). We have 10 such queues, each handling a different category of work. If a job fails (e.g. email server is down), it retries automatically up to 3 times.

10 queues, each with a dedicated worker process (or in-process in production):

Queue	Trigger	What worker does
email	OTP, welcome, password reset	Sends via Resend API
notification	Post reactions, comments, mentions	Creates DB notification rows
notification-digest	Daily cron	Emails unread notification summaries
badge	Points earned, milestones hit	Awards badges, updates leaderboard
group-digest	Weekly cron	Sends group activity summary email
mentorship	Request accepted or 48h idle	Reminder emails + 7-day auto-expiry
push	Notification created	Web Push fan-out to all user subscriptions
content-sync	Admin triggers	WordPress fetch → Skyvern fallback
feed-ranking	Hourly cron	Recomputes hot_score on all posts
post-lifecycle	Every minute cron	Flips scheduled→published, published→archived

5.4 Design System

Rather than every developer picking their own colours and spacing, we defined one set of named values — "design tokens" — in a single CSS file. Every component in the app uses names like `var(--uc-orange)` instead of writing the hex code `#F05A28` directly. This means: (1) if we ever want to rebrand, we change one file and it updates everywhere, (2) dark mode and light mode are just two sets of values for the same names, swapped by a `data-theme` attribute on the body. The default theme is "Warm Futuristic Dark" — a deep navy background with UIU's orange as the identity colour and indigo for interactive elements. We enforced strict rules: no raw hex values in components, pill-shaped buttons everywhere, sentence case text, no heavy font weights — so the UI looks consistent no matter who wrote the component.

Single source of truth: `apps/web/src/styles/tokens.css`

```
Two themes: Warm Futuristic Dark (default) + Warm Neutral Light (data-theme='light')
Brand colors:
  --uc-orange: #F05A28  (UIU identity)
  --uc-indigo: #5B5BD6  (interactive elements)
  --uc-navy:   #1E3A70  (accent surfaces)

Surface stacking (no box-shadow, depth via background):
  --surface-page   #060D1A  (outermost)
  --surface-card   #0A1628  (cards)
  --surface-raised #111D35  (modals, dropdowns)

Rules enforced in every component:
- No hardcoded hex — always var(--token-name)
- Border: 0.5px solid var(--border-*) — never 1px structural
- Buttons: border-radius: var(--r-pill) — no sharp corners
- Font weight: 400 and 500 only — never 600+
- Sentence case everywhere — no ALL CAPS
```

5.5 Driver Role & Shuttle System

Students want to know where the university shuttle is right now. The app has a live map that shows the bus moving along its route in real time. Here's how it works: the driver opens the app on their phone, it picks up their GPS location every 5 seconds and sends it to the server. The server broadcasts this to all students watching the shuttle page. The bus icon moves smoothly on the map. Drivers have a

special "driver" role that can only do one thing — broadcast GPS coordinates. They cannot post to the feed, message students, or access anything else. This is the principle of least privilege: you only get access to exactly what you need for your job.

A minimal-privilege role added specifically for transport staff.

Migration 073:

```
users.role enum gains 'driver'  
shuttle_routes gains: est_duration_min, cycle_minutes (client-side interpolation params)
```

Driver flow:

```
ShuttleDrivePage.tsx → navigator.geolocation.watchPosition()  
→ POST /shuttle/locations every 5s { route_id, lat, lng }  
→ only 'driver' role can call this endpoint (requireRole('driver'))  
→ socket emits SHUTTLE_LOCATION to uni:{universityId} room  
→ ShuttlePage.tsx receives live coords, interpolates bus position on route  
  using est_duration_min + cycle_minutes without needing a real GPS server
```

6. Database Design

81 migrations building a schema from scratch. Key design decisions:

```
UUID primary keys:  id UUID DEFAULT uuid_generate_v4()
Timestamps:         created_at / updated_at TIMESTAMPTZ DEFAULT now()
Soft deletes:       is_deleted BOOLEAN DEFAULT false    (never deleted_at)
Indexes:            every FK column + (university_id, created_at DESC) on high-volume tables
JSONB columns:      user_settings.notification_preferences, user_settings.privacy_preferences
Generated columns:  search_vector TSVECTOR GENERATED ALWAYS AS (...) STORED    (migration 070)
No Postgres RLS:    all isolation via service-layer WHERE university_id = ?
```

Key schema groups:

Group	Tables
Core	universities, university_settings, users, profiles, invitations, user_sessions
Feed	posts, comments, reactions, saved_posts, tags, post_tags, polls, poll_votes
Jobs	jobs, job_applications
Events	events, event_rsups
Groups	groups, group_members, group_join_requests, group_resources, group_study_sessions, group_session_rsups
Messaging	conversations, conversation_participants, messages, message_reads
Notifications	notifications, news
Mentorship	mentorship_requests, mentorship_sessions, mentorship_feedback, gift_cards, mentorship_redemptions
Connections	connections
Profile	profile_experiences, profile_education, profile_featured, profile_views
Campus	lost_found_items, shuttle_routes, shuttle_schedules, shuttle_locations
Push	push_subscriptions
Settings	user_settings
Sync	content_sync_runs, content_attachments
Audit	university_audit_log
Moderation	user_moderation, account_deletion_requests

7. Teacher Q&A Bank

Architecture & Design

Q: Why did you choose a monorepo structure?

A: We have three packages — `apps/web`, `apps/api`, and `packages/shared`. The shared package contains Zod schemas and TypeScript types that both the frontend and backend use. Without a monorepo, we'd have to publish the shared package to npm or duplicate types. With pnpm workspaces and a monorepo, a single `import { createPostSchema } from '@unicconnect/shared'` works in both apps. This eliminates schema drift between client and server validation.

Q: How does multi-tenancy work? Can University A see University B's data?

A: No. Every table has a `university_id` UUID foreign key. The `resolveUniversity` middleware runs on every request, resolves the university from the `x-university-domain` header, and sets `req.university.id`. Every service call then filters by that ID — `db('posts').where({ university_id: universityId })`. We never read `university_id` from the request body. There is no Postgres Row-Level Security; we chose application-level isolation for simpler debugging.

Q: Why not use an ORM like Prisma or TypeORM?

A: We use Knex, which is a query builder, not an ORM. Knex gives us direct control over SQL, handles migrations cleanly with a numbered file system, and doesn't add the weight of ORM entity mapping. Since we have complex multi-tenant queries with dynamic `university_id` filters and full-text search using Postgres-specific features like `tsvector`, raw SQL proximity is valuable. We drop to raw SQL only when Knex genuinely can't express the query.

Q: How do you ensure the frontend never calls the wrong university's API?

A: The Vite frontend has an environment variable `VITE_UNIVERSITY_DOMAIN=uiu.ac.bd`. The Axios instance (in `apps/web/src/lib/axios.ts`) automatically injects `x-university-domain: uiu.ac.bd` on every HTTP request. On the server, the middleware reads this header and resolves the university — never from any user-controlled body parameter.

Authentication & Security

Q: How are JWT tokens stored? Isn't localStorage insecure?

A: The access token (15 min lifetime) is stored in Zustand `authStore` — memory only. It lives in JavaScript heap, not `localStorage`, so it's not accessible to XSS scripts reading `localStorage`. The refresh token (7 days) is an `httpOnly` cookie, which JavaScript cannot read at all. `localStorage` stores only a sentinel flag `uc:has_session` — a boolean, not a token — so the axios interceptor knows whether to attempt a silent refresh on 401.

Q: What happens when the access token expires?

A: The axios interceptor catches every 401 response. If the session sentinel exists in localStorage, it calls `POST /auth/refresh` — the browser automatically sends the httpOnly refresh cookie. The server validates the refresh token against `user_sessions` table (prevents reuse), issues a new access token, and the interceptor retries the original request. The user never notices. If the refresh also fails (e.g. token revoked), the user is logged out.

Q: Why do you use an invite token for registration?

A: UniConnecT is a private network. Students can't self-register — they need an invite from an admin. This prevents random external users from joining. The invite contains a pre-set email and role, so a student can't register with an alumni invitation. Admins can bulk-create invitations via the admin panel.

Q: How does OTP work?

A: After register, the server generates a 6-digit OTP, stores it in Redis with a 10-minute TTL keyed to the `userId + purpose` (e.g. `otp:email_verify:{userId}`), and sends it via email (Resend API → email queue → email worker). `POST /auth/verify-otp` checks the Redis key, marks the user verified, and deletes the key. Redis ensures OTPs expire automatically without a cleanup cron.

Q: What happens if a user registered but never verified their email, then tries to log in later?

A: Before this fix, they would be stuck — the OTP from registration had already expired (10-minute TTL in Redis) and the login page just showed a generic error with no way forward. Now, when the server sees the password is correct but `is_verified = false`, it immediately revokes the old expired OTP, generates a fresh 6-digit code, emails it to the user, and responds with `ACCOUNT_NOT_VERIFIED` (403). The frontend catches that specific error code and navigates the user straight to the OTP verification page with their email pre-filled. They verify the new code and are fully activated. No admin intervention needed.

Q: What if an admin deletes an account and that person tries to re-register?

A: It depends on whether the deleted account was ever verified. If the old account was verified (a real person who used the platform), we block re-registration with `ACCOUNT_DEACTIVATED` (409) and tell them to contact admin — their data exists and needs admin review before restoration. If the deleted account was never verified (a ghost/abandoned invite), we hard-delete that leftover row entirely and let the re-registration proceed cleanly. Without this check, the registration would crash with a silent 500 error because the database's global unique index on email would reject the duplicate, even for soft-deleted rows.

Q: Can a user reactivate their own deactivated account?

A: Yes, if they deactivated it themselves. When a user deactivates their account via Settings, we set `is_active = false` AND record `deactivated_at = now()`. That timestamp is the signal it was self-initiated. If they log in again later with the correct password, the server sees `deactivated_at` is set, understands it was voluntary, and automatically reactivates the account — no admin needed. If an admin disabled the account, `deactivated_at` stays NULL, so the same login returns the generic 401 — account stays blocked until admin re-enables it.

Q: How are passwords stored?

A: Using bcrypt with a cost factor of 12. We never store plaintext passwords. The JWT payload contains only { `userId`, `universityId`, `role` } — no sensitive data. The `JWT_SECRET` and `JWT_REFRESH_SECRET` are separate secrets, so a refresh token cannot be used as an access token.

Q: How do you protect the API from abuse / spam?

A: A Redis-backed rate limiter (`rateLimiter.ts`) guards every mutating route. It uses a sliding counter keyed by `rl:{prefix}:{subject}` where the subject is the authenticated user id (`u:{userId}`) when available, falling back to the IP for anonymous routes like login. Keying by user id is deliberate — an entire university sitting behind one NAT IP would otherwise be throttled as a single client. Over the limit returns HTTP 429. It's a no-op in the test environment so tests aren't throttled.

Q: How can users protect themselves from harassment?

A: The moderation module gives users two tools plus reporting. **Block** (`POST /moderation/block/:userId`) hides the user both ways — no posts, no DMs in either direction. **Mute** silences a user one-directionally — you stop seeing their content but they aren't notified. **Report** files a content or user report that lands in the admin review queue. All three are backed by the `user_moderation` table (migration 078), and admin resolutions are written to `university_audit_log`.

Real-time System

Q: How does Socket.io scale across multiple server instances?

A: Socket.io uses a Redis pub/sub adapter. We create two `IORedis` connections — `pubClient` and `subClient` — and pass them to `@socket.io/redis-adapter`. When any server instance emits `io.to('uni:abc').emit(...)`, the adapter publishes to Redis; the adapter on all other instances receives it and forwards to connected sockets in that room. This allows horizontal scaling without sticky sessions.

Q: How does the client know when to reconnect after a token expires?

A: The Socket.io middleware on the server calls `tokenService.verifyAccessTokenWithExpiry(token)`. If the token is expired (not just invalid), it sends back `TOKEN_EXPIRED` error. The client socket listens for connection errors, detects `TOKEN_EXPIRED`, silently refreshes the access token via the HTTP interceptor, then reconnects with the new token.

Q: What are Socket.io rooms used for?

A: Three room types: `uni:{universityId}` for university-wide broadcasts (new posts, events, shuttle locations), `user:{userId}` for personal events (notification received, connection request, direct message), and `conv:{conversationId}` for chat messages. When a socket connects, it automatically joins its university room and personal room. It joins conversation rooms only when actively viewing a conversation.

Background Jobs

Q: Why use Bull queues instead of just `await` on async operations?

A: HTTP handlers must respond quickly. If sending an email takes 2 seconds, the user waits. With Bull, we enqueue the job (milliseconds), respond to the user, and the worker processes it asynchronously. Bull also provides retry logic, exponential backoff, job persistence in Redis (survives server restart), concurrency control, and job monitoring. A failed email automatically retries three times before giving up.

Q: What happens if the Bull worker crashes mid-job?

A: Bull marks the job as `active` when a worker picks it up. If the worker crashes without completing, Bull moves the job back to `waiting` after a stall timeout. On restart, the worker picks it up again. This is why our workers are idempotent — re-running them doesn't duplicate emails or notifications.

Q: How does the feed `hot_score` work?

A: We adapted the Hacker News algorithm. $hot_score = (reactions + comments \times 2) / (age_hours + 2)^{1.8}$. The denominator grows as the post ages, so older posts decay. Reaction and comment counts are maintained transactionally on every write (not computed at read time). The `feed-ranking` Bull cron runs hourly and updates `hot_score`. The "Top" feed then sorts by this pre-computed column using a GIN/btree index — no subquery aggregation at read time.

Database

Q: Why are there two migration 072 files?

A: This was a mistake made during parallel development — two features (`post_lifecycle` columns and `username` column) were given the same migration number. Knex runs migrations by filename, so both run, and they don't conflict functionally. Going forward, we start from `077_` to avoid re-using any prefix. The lesson: coordinate migration numbering in the team's task tracker before branching.

Q: Why use generated stored columns for full-text search instead of a separate search index?

A: Generated STORED columns compute the `tsvector` value at write time (INSERT/UPDATE) and persist it in the row. The GIN index is built on this stored value. At read time, the query `WHERE search_vector @@ plainto_tsquery('english', ?)` uses the index — $O(\log n)$ not $O(n)$. Alternatives like Elasticsearch add operational complexity (separate cluster, sync pipeline). For our scale, Postgres full-text search is sufficient and has zero additional infrastructure cost.

Q: Why don't you use Postgres Row Level Security (RLS)?

A: RLS would automatically filter by university at the database driver level, which is more secure. We chose application-level filtering because: (1) RLS requires setting a session variable per request which complicates connection pooling, (2) debugging RLS policies is harder than reading service code, (3) our service layer is consistent — every query includes `where({ university_id })`. For a production system at larger scale, RLS would be the next evolution.

Frontend

Q: Why TanStack Query instead of Redux for server state?

A: Redux is general-purpose state — you have to manually handle loading, error, caching, and refetching. TanStack Query is built specifically for server state. It gives us automatic background refetch, stale-while-revalidate caching, pagination, optimistic updates, and request deduplication out of the box. We use Zustand only for truly global client state (auth token, socket connection, theme). The rule is: server data → TanStack Query, UI state → Zustand.

Q: How are routes protected? Can an unauthenticated user access the feed?

A: Route components are wrapped in `ProtectedRoute`, `AdminRoute`, or `GuestRoute` (from `apps/web/src/router/`). `ProtectedRoute` checks `authStore` for a valid access token. If none, it redirects to `/login`. `AdminRoute` additionally checks `role === 'admin' || role === 'faculty'`. `GuestRoute` redirects logged-in users away from login/register pages.

Q: Why are all pages lazy-loaded?

A: `React.lazy + Suspense` means each page is a separate code chunk loaded only when navigated to. The initial bundle is small — users don't download the admin panel code if they never visit it. The `page()` helper in `apps/web/src/router/index.tsx` wraps `React.lazy + Suspense` with a loading fallback so every page automatically has a loading state.

Q: How does the real-time notification badge work?

A: The `notificationsStore` (Zustand) holds an `unreadCount`. On login, it fetches the count from `GET /notifications/unread-count`. The socket listens on the `user:{userId}` room for `NOTIFICATION_NEW` events — each increments `unreadCount` in the store. The `TopNav` badge reads this store. When the notifications page is opened, it calls `POST /notifications/read-all` and resets the count.

Deployment & DevOps

Q: How is the application deployed?

A: The backend API is deployed to Azure App Service via GitHub Actions CI/CD triggered on pushes to `main`. The production start command runs `npx pnpm --filter api db:migrate:prod` before starting the server, so migrations apply automatically on each deployment. The frontend is deployed to Vercel (configured via `vercel.json`). The database is hosted on Neon (managed Postgres with branching).

Q: How do Bull workers run in production?

A: In development, workers run as a separate process (`pnpm --filter api worker`) to avoid blocking the HTTP server. In production on Azure App Service (single process constraint), workers run in-process with the API server — the

`workers/index.ts` barrel registers all workers. The queues use Redis (same instance) for durability regardless of whether workers are in-process or separate.

Q: What happens if the database migration fails mid-deploy?

A: Knex tracks completed migrations in a `knex_migrations` table. Each migration is a single transaction — if it fails, it rolls back. The server startup fails early and Azure redeploys to the previous version. We learned this the hard way: migration `070` added a `STORED GENERATED` column using `array_to_string()` which is only `STABLE`, not `IMMUTABLE`. Postgres requires `IMMUTABLE` functions in stored generated columns. The fix was to use `to_tsvector()` directly (which is immutable) instead of `array_to_string()`.

Q: How do you manage environment variables across environments?

A: `apps/api/.env` (local, not committed), `apps/web/.env` (local). Production secrets are managed via Azure App Service configuration for the API and Vercel environment variables for the web. The `.env.example` file documents every required variable. There is a strict separation: `JWT_SECRET` and `JWT_REFRESH_SECRET` are separate so that a compromised refresh token can't be used as an access token.

Feature-specific

Q: How does the content sync avoid re-importing the same article twice?

A: Each imported row carries an `external_id` (the WordPress post ID). Before inserting, the worker queries `WHERE university_id = ? AND external_id = ?`. If the row exists, it's skipped (idempotent). This means running the sync twice never duplicates content. The `content_attachments` table similarly deduplicates by `(university_id, source_url, attachable_type, attachable_id)`.

Q: What is the Skyvern fallback in content sync?

A: Some university websites don't expose a WordPress REST API — they're plain HTML pages. When the direct WordPress fetch fails, `content-sync.worker.ts` calls `skyvern.service.ts`, which uses Skyvern's browser automation API to navigate to the URL, extract the DOM, and return structured content. Skyvern runs a real Chromium browser, so it can scrape JavaScript-rendered pages. This is significantly slower (30–60s vs 1–2s for REST) so it's a fallback only.

Q: How does the lost-and-found feature work?

A: `lost_found_items` table with fields: `type` (lost/found), `title`, `description`, `category`, `location`, `image_url`, `status` (open/claimed), `reporter_id`. Users post items they lost or found. The campus module's routes are at `/api/v1/campus/lost-found`. Claimants contact the reporter via the messaging system. No automated matching — human-driven.

Q: How does the shuttle GPS work for students watching the map?

A: `ShuttlePage.tsx` subscribes to the `SHUTTLE_LOCATION` socket event on the `uni:{universityId}` room. When a driver broadcasts their GPS location, all students on the page receive it. Since GPS updates are every 5 seconds and sockets have some latency, the frontend uses `est_duration_min` and `cycle_minutes` (stored on the route in migration 073) to linearly interpolate the bus position between updates, giving smooth animation.

Q: Can a user delete their account?

A: Not directly. Users submit an account deletion request with a reason (`POST /users/me/account-deletion-request` — migration 079). Admins review requests in the admin panel and can approve or deny. This design (requested vs self-service deletion) is intentional for a university setting where student records may have administrative obligations. Users can deactivate their account (`POST /users/me/deactivate, sets deactivated_at`) which hides them from other users immediately.

Q: How do drafts work across different content types?

A: Migration 064 added `is_published` + lifecycle columns to posts, jobs, news, and events tables. `GET /me/drafts` is a UNION query across all four tables, filtered by `author_id = me AND is_published = false`. The result is a unified list regardless of content type. This is the `drafts` module — a thin read-only endpoint that surfaces unsaved work across the entire platform.

Q: How do new users know what to do first (onboarding)?

A: `GET /users/me/progress` returns a set of first-run signals validated by `profileProgressSchema` in `packages/shared` — a weighted `profileScore` (0–100), plus granular booleans (`hasAvatar`, `hasBio`, `hasHeadline`, `hasAddedExperience`, `hasAddedEducation`, `hasMadePost`, `isVerified`) and `connectionCount`. The `OnboardingChecklist` component reads these and deep-links the user straight to the exact gap — for example "Add a bio" opens the edit-profile modal. The checklist disappears once `profileScore` reaches 100. The flags are computed server-side from the actual profile so they can never drift from reality.

Testing

Q: How are your integration tests structured?

A: `apps/api/src/__tests__/setup.ts` runs all pending Knex migrations against a test database (separate `TEST_DATABASE_URL`), seeds four canonical users (admin, faculty, alumni, student) with known credentials, and exports a `loginAs(email, password)` helper that returns `{ accessToken, cookie }`. Each test file uses `loginAs()` to get bearer tokens. All supertest requests must include `.set('x-university-domain', DOMAIN)`. Tests clean up in `afterEach/afterAll` hooks.

Q: Do you have frontend tests?

A: Yes — React Testing Library + Vitest + MSW (Mock Service Worker). MSW intercepts HTTP requests at the network level (not axios mocks) so tests are framework-agnostic and test behavior rather than implementation. Handlers live in `src/tests/msw/handlers.ts`. Test files are co-located with their components as `*.test.tsx`.

Quick Reference — Who Owns What

If the teacher asks about...	Talk to...
How the JWT / refresh token / auth flow works	Joydip Datta (Leader)
Why we chose this architecture / monorepo	Joydip Datta (Leader)
How Socket.io scales / real-time events	Joydip Datta (Leader)
Bull queues, background jobs, workers	Joydip Datta (Leader)
Admin panel, user management, audit logs	Joydip Datta (Leader)
File upload, presigned S3 URL pattern	Joydip Datta (Leader)
Rate limiting / API abuse protection	Joydip Datta (Leader)
Moderation (block / mute / report)	Joydip Datta (Leader)
Account deletion request flow	Joydip Datta (Leader)
Drafts (how posts/jobs/events share drafts)	Joydip Datta (Leader)
Database design decisions (why UUID, no RLS)	Joydip Datta (Leader)
Testing strategy, CI/CD, deployment	Joydip Datta (Leader)
Design system, tokens, dark/light theme	Joydip Datta (Leader)
How the feed works, hot_score algorithm	Md. Saem Ferdous (2)
Post scheduling / draft / lifecycle states	Md. Saem Ferdous (2)
Groups, join requests, member roles	Md. Saem Ferdous (2)
Notifications, news, badges, share	Md. Saem Ferdous (2)
Jobs, events, RSVP	Md. Mahfujur Rahman Himel Akon (3)
Connections graph, mutual connections	Md. Mahfujur Rahman Himel Akon (3)
Profile sections, experience, education	Md. Mahfujur Rahman Himel Akon (3)
Onboarding checklist, profile progress	Md. Mahfujur Rahman Himel Akon (3)
Mentorship, points economy, gift cards	Md. Mahfujur Rahman Himel Akon (3)
Chat messages, conversation types	Md. Monabbur Hosen Bhuiyan (4)
Presence system, online status	Md. Monabbur Hosen Bhuiyan (4)
Web Push notifications, VAPID	Md. Monabbur Hosen Bhuiyan (4)
Privacy settings, visibility tiers	Md. Monabbur Hosen Bhuiyan (4)
Full-text search, GIN indexes	Md. Nazmul Hasan Nasim (5)
Content sync, WordPress, Skyvern	Md. Nazmul Hasan Nasim (5)
Shuttle GPS, lost-and-found, explore	Md. Nazmul Hasan Nasim (5)
Settings, notification / privacy preferences	Md. Nazmul Hasan Nasim (5)

